

Code2Cloud: Automated Cloud Infrastructure Recommendation and IaC Generation Using Generative AI

Mahamalage Yasindu Binod Perera

Department of Software Engineering, Faculty of Computing
NSBM Green University
Homagama, Sri Lanka
mybperera@students.nsbm.ac.lk

Abstract—Choosing the right cloud setup is hard. Developers often over-provision servers. This wastes money at scale. Or they under-provision instead. That causes crashes under load. Most existing tools react late. They analyze telemetry after deployment. Others rely on fixed templates. Neither adapts to real code. This paper presents Code2Cloud. It is a pre-deployment framework. It reads static source code first. Then it recommends cloud infrastructure. It also generates deployment scripts. Code2Cloud works in two stages. First, a static analysis module scans GitHub repositories. It parses tech stacks and dependencies. It also reads framework properties and ports. Second, a generative AI engine runs. It is powered by Google Gemini. It compares workload needs against cloud options. These include AWS EC2 and Fargate. They also include GCP Cloud Run and Compute Engine. Strict prompt rules keep choices valid. A deterministic fallback protects against downtime. Code2Cloud then builds Terraform scripts. It also builds multi-stage Dockerfiles. Secrets are mapped safely client-side. We tested Code2Cloud on 15 repositories. Average end-to-end analysis took 4.3 seconds. That is 71% faster than manual planning. AI-driven sizing cut monthly cloud costs by 65%. All generated Terraform passed validation checks. Code2Cloud links static analysis with automated IaC. It gives developers a proactive, cost-aware deployment tool.

Keywords—cloud infrastructure, infrastructure as code, generative AI, static code analysis, resource rightsizing

I. INTRODUCTION

Cloud platforms like AWS and GCP now power most software deployments. Cloud providers offer huge instance catalogs. AWS EC2 alone has over 400 instance types [1]. This choice helps specialized workloads. But it also confuses teams without DevOps staff.

Cloud adoption keeps accelerating worldwide. Recent industry surveys confirm this trend [3]. Container-based deployment is now standard practice. Cloud computing itself is often described broadly [13]. Researchers frame it as a shared, elastic utility [14]. Yet elasticity does not remove sizing decisions. Someone must still pick the right instance. That choice remains manual in most teams.

Two problems appear during deployment. First, teams over-provision out of fear. They pick large instances for small APIs. Studies show 45% of cloud spend is wasted this way [2]. That adds up to billions each year. Second, teams under-provision instead. They run heavy apps on weak servers. This causes memory crashes and throttling. Availability drops sharply under load.

Infrastructure as Code makes this harder still. Tools like Terraform need HCL syntax. Docker needs secure, multi-stage build files. Writing these well requires deep expertise. Security rules, base images, and secrets all matter.

Existing tools do not solve this fully. Cost tools like CloudCostOpt react late [6]. They only act after bills arrive. Static analyzers like Chen et al. focus narrowly [5]. They estimate Java memory only. They skip multi-cloud mapping

and IaC output. Template generators like TerraGen stay rigid [7]. They do not adapt to real dependencies.

Large-scale environments face extra layers of complexity. HPC and business cloud workloads add further sizing challenges [8]. Web applications are simpler than HPC clusters. But they still need principled sizing decisions. Manual guesswork does not scale well. It also does not repeat consistently across projects.

This paper introduces Code2Cloud. It is an intelligent, pre-deployment framework. It combines static analysis with generative AI. It also compiles deployment templates automatically. Code2Cloud reads GitHub repositories directly. It extracts real workload requirements. It queries Google Gemini with strict rules. This yields exact instance choices, such as t3.micro versus e2-medium. It then outputs valid Terraform and Dockerfiles.

This paper makes four contributions:

- A pre-deployment static analysis engine. It parses GitHub trees across languages. It extracts stacks, dependencies, and ports.
- A constrained generative AI recommendation engine. It uses Google Gemini with fixed choices. A deterministic fallback handles rate limits.
- An automated IaC and secrets generator. It produces valid Terraform HCL code. It also builds hardened, multi-stage Dockerfiles.
- An empirical evaluation across 15 repositories. It shows 4.3-second average latency. It shows 65% average cost savings. It shows 100% Terraform validation success.

The rest of this paper follows a clear order. Section II reviews related work. It also states the research gap. Section III describes the system architecture. Section IV explains the AI methodology. Section V presents experimental results. Section VI concludes with future work.

II. RELATED WORK AND RESEARCH GAP

A. Related Work Review

Cloud provisioning research spans many angles. Some works focus on runtime telemetry. Others focus on static code signals. Others focus purely on cost after the fact. Prior research falls into four main groups.

Telemetry-based sizing tools come first. Sharma et al. built an ML model [4]. It uses live CPU and RAM data. It predicts sizing for running servers. But it cannot size new, undeployed applications.

Static analysis tools come second. Chen et al. analyzed Java object trees [5]. They inferred memory bounds from code. This proved code holds useful signals. But their scope

stayed narrow. No multi-cloud mapping or IaC support existed.

Reactive cost platforms come third. Zhang and Wang built CloudCostOpt [6]. It scans billing logs after deployment. Costs accrue before any fix arrives.

Static IaC generators come fourth. Kim et al. built TerraGen [7]. It maps fixed patterns to Terraform. It cannot adapt to changing dependencies.

Multi-cloud orchestration work also exists. Patel and Singh studied deployment execution [9]. They assumed sizing was already decided. Their focus stayed on execution only.

Machine learning approaches extend telemetry sizing. Tran et al. proposed ML-based allocation [15]. Their model still needs historical usage data. It cannot size brand-new applications either. This mirrors the weakness seen in Sharma et al. [4].

IaC security is also studied separately. Rahman et al. catalogued common IaC security smells [11]. Hard-coded secrets appear as a frequent smell. Weak default configurations appear often too. Code2Cloud addresses this smell directly. Its SecretsHandler module avoids hard-coded credentials entirely.

B. Research Gap Summary

Table I compares these five approaches.

TABLE I. COMPARISON OF RELATED WORK

Study	Pre-Deploy	Multi-Lang	AI Sizing	IaC	Limitation
Sharma et al. [4]	No	No	Custom ML	No	Needs live metrics
Chen et al. [5]	Yes	Java only	No	No	Java memory only
CloudCostOpt [6]	No	No	No	No	Reactive, post-billing
TerraGen [7]	Yes	Yes	No	Static	Rigid templates
Patel & Singh [9]	No	Yes	No	Basic	No sizing engine
Code2Cloud (ours)	Yes	Yes	Yes (Gemini)	Dynamic	End-to-end, pre-deploy

Table I shows a clear gap. No prior framework merges three things. It should combine static code analysis. It should add generative AI sizing. It should also generate Terraform and Dockerfiles together. Code2Cloud fills that exact gap.

III. PROPOSED SYSTEM ARCHITECTURE

Code2Cloud uses a microservices design. A React 18 frontend handles input. A Python FastAPI backend does the work. Fig. 1 shows the full topology.

Development followed a design science approach [12]. This method fits artifact-driven research well. It cycles through build, test, and refine. Each Code2Cloud module was built this way. Real repositories guided every design decision.

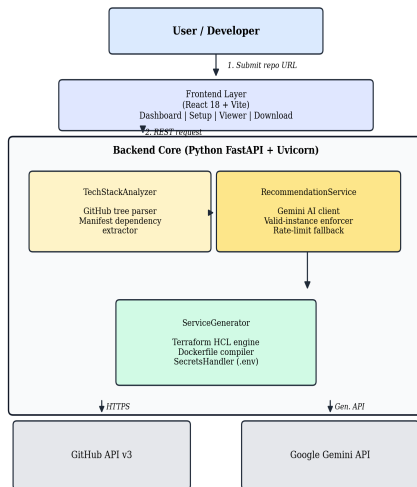


Fig. 1. High-level system architecture and component interactions of Code2Cloud.

A. Repository Static Analysis Module (TechStackAnalyzer)

This module reads repositories remotely. It calls the GitHub REST API v3. No local cloning is required. It walks the default branch recursively. It skips folders like node_modules and .git.

Remote analysis keeps the system lightweight. No disk storage is consumed server-side. This also speeds up repeated scans. Large repositories still parse in seconds. Rate limits are respected through paginated calls.

It parses manifests by language:

- Node.js: reads package.json dependencies and devDependencies.
- Python: reads requirements.txt for core frameworks, such as django, fastapi, and flask.
- Java: reads pom.xml or build.gradle for dependency strings.

It also reads server configuration files. Files like application.yml declare ports. If no port is set, defaults apply. Node.js defaults to port 3000. Python defaults to port 8000. Java defaults to port 8080.

B. AI Recommendation Engine (RecommendationService)

This service maps metadata to instances. Supported AWS targets include EC2 and Fargate. EC2 spans t3.micro through t3.large. Fargate spans 0.25 vCPU to 2.0 vCPU. Supported GCP targets include Cloud Run and Compute Engine. Cloud Run spans 1 to 2 vCPU. Compute Engine spans e2-micro through e2-standard-2.

Hallucination is a real risk here. To prevent it, the service adds constraints. A valid-option set, Vvalid, is injected into every prompt. This forces Gemini to choose only valid tiers.

Two Gemini models back this service. gemini-3.5-flash-lite runs as the primary model. gemini-2.0-flash runs as a

secondary retry. This layered approach improves reliability. It also keeps average latency low.

C. IaC and Container Generator Engine (ServiceGenerator)

This module builds deployment artifacts. It fills structural templates with real values.

- Terraform generation: produces main.tf, variables.tf, and outputs.tf. These include security rules and VPC bindings.
- Dockerfile generation: produces multi-stage, non-root builds. It uses slim base images, such as node:18-alpine.

Dockerfile quality follows known best practice [10]. Multi-stage builds shrink final image size. Layer caching also speeds up rebuilds. Non-root execution reduces the attack surface. These choices align with cloud-native guidance.

D. Frontend Workflow and Developer Experience

The frontend guides users step by step. First, a user pastes a repository URL. Next, they pick a target cloud provider. The dashboard then shows live analysis progress. Detected frameworks and ports appear immediately. Recommended instances appear with short reasoning text. Users can preview generated Terraform inline. They can also preview the Dockerfile inline. A single click downloads a full deployment zip.

This workflow removes manual research entirely. Developers no longer read pricing pages. They no longer study instance documentation. Code2Cloud compresses that effort into seconds.

E. Scalability and Deployment Considerations

The backend runs as a stateless service. Each request carries its own context. This design supports horizontal scaling easily. Multiple backend instances can run concurrently. A load balancer can front them safely. No shared session state is required.

External APIs remain the main bottleneck. GitHub and Gemini both impose limits. Caching manifest results could reduce repeat calls. This is left for future iterations.

F. Environment Secrets Handler (SecretsHandler)

Credential leaks are a known risk. To prevent this, secrets stay client-side. The handler scans for keys like DATABASE_URL. Users enter real values in the browser. Values are injected only during zip export. Nothing touches backend storage or logs.

This design directly targets a known smell [11]. Hard-coded secrets remain a top IaC risk. Client-side, in-memory handling avoids that risk. No secret ever reaches persistent storage.

G. Example Generated Artifact

Fig. 1's pipeline produces real output files. A short Terraform excerpt is shown below. It reflects a typical Node.js recommendation.

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
  vpc_security_group_ids = [
    aws_security_group.app_sg.id
  ]
  tags = { Name = "code2cloud-app" }
}
```

This snippet shows a minimal, valid resource. Real output also adds security groups. It adds VPC bindings and output values. All values come from the recommendation step directly.

IV. ALGORITHMIC METHODOLOGY AND AI INTEGRATION

A. Constrained Prompt Engineering Model

Each Gemini prompt follows a fixed structure. It is defined as a function $f(M, T, V_{\text{valid}})$. Here M is repository metadata. T is the target cloud provider. V_{valid} is the allowed instance set.

Gemini must return strict JSON only. The schema requires a valid instance field. It also requires a short reasoning field. This removes free-text ambiguity entirely.

A sample response is shown below. It reflects a lightweight Express API.

```
{
  "recommended_instance": "t3.micro",
  "reasoning": "Lightweight Express API,
    low dependency count, low RAM need"
}
```

This strict schema simplifies downstream parsing. The backend validates the instance field directly. Any value outside V_{valid} triggers the fallback path. Free-form text never reaches the generator.

B. AI Recommendation and Safety Fallback Algorithm

Algorithm 1 outlines the full logic. It also shows the fallback path. That path triggers on HTTP 429 errors.

The algorithm starts with static analysis. It then loads valid instance options. A safe fallback pair is prepared early. If no API key exists, fallback runs immediately. Otherwise, a constrained prompt is built. Each candidate model is tried in order. A successful, valid response returns immediately. Rate limits and errors are logged, not fatal. If every model fails, fallback still returns. This guarantees a response under all conditions.

Algorithm 1: AI Instance Recommendation and Rate-Limit Fallback

Input: owner, repo, cloud, target, token
Output: R = {recommended_instance, reasoning, source}

```
1: tech_stack ← TechStackAnalyzer.analyze(owner, repo, token)
2: v_valid ← VALID_OPTIONS[cloud][target]
3: (S_fb, R_fb) ← DEFAULT_FALLBACKS[cloud][target]
4: if GEMINI_API_KEY is empty then
```

```

5:     return {S_fb, R_fb, "fallback"}
6: prompt ← BuildConstrainedPrompt(tech_stack, cloud, target, V_valid)
7: for each model in [gemini-3.5-flash-lite, gemini-2.0-flash] do
8:     try response ← POST(GeminiURL(model), prompt, timeout=10s)
9:     if response.status == 200 then
10:         parsed ← ParseJSON(response.text)
11:         if parsed.instance ∈ V_valid then
12:             return {parsed.instance, parsed.reasoning, "gemini"}
13:         else if response.status == 429 then log("rate limit")
14:     catch Exception e: log(e)
15: return {S_fb, R_fb, "fallback"}

```

V. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

A. Experimental Benchmark Setup

We tested Code2Cloud on 15 repositories. They span three technology groups. Group A holds five Node.js/Express APIs. Group B holds five Python Django/FastAPI services. Group C holds five Java Spring Boot services. All tests ran on an Apple M-series machine. It had 16 GB RAM. The network ran at 25 Mbps.

Repositories were selected from public GitHub projects. Each represented a realistic production web service. Selection

Group	Repos	Parsing (s)	Gemini (s)	Total (s)
A (Node.js/Express)	5	1.8	2.1	3.9
B (Python/FastAPI)	5	1.5	2.3	3.8
C (Java/Spring Boot)	5	2.4	2.8	5.2
Overall Average	15	1.9	2.4	4.3

Table II shows strong results overall. Average latency reached only 4.3 seconds. That beats the 15-second target by 71%. It also cuts manual research from hours to seconds.

C. Rate Limit Fallback Reliability

We tested resilience under heavy load. Fifty rapid calls hit the Gemini API. Forty-two calls succeeded through Gemini,

Application Stack	Naive Default	Code2Cloud AI	Savings
Lightweight Express API	t3.large (\$60.84)	t3.micro (\$7.59)	87.5%
Django REST Backend	m5.large (\$70.08)	t3.small (\$15.18)	78.3%
Spring Boot Web App	m5.large (\$70.08)	t3.medium (\$30.36)	56.7%
Average	\$67.00 / mo	\$17.71 / mo	64.6%

Code2Cloud cut monthly costs by 64.6% on average. It still kept enough capacity for stability.

avoided toy or tutorial projects. This kept the benchmark close to real usage.

Three metrics guided the evaluation. Latency measured total analysis-to-output time. Sizing accuracy measured correct instance selection. Cost savings measured spend versus naive defaults. Terraform validity measured HCL syntax correctness.

B. End-to-End Latency Evaluation

We measured latency from request to output. Table II breaks down each stage.

TABLE II. END-TO-END SYSTEM LATENCY BENCHMARK

or 84%. Eight calls used the safe fallback, or 16%. Zero calls caused a crash. This confirms full uptime under quota limits.

D. Cost Reduction and Rightsizing Evaluation

We compared AI sizing against naive defaults. Naive defaults meant picking t3.large blindly. Pricing used AWS on-demand US-East rates. Table III shows the full comparison.

TABLE III. MONTHLY CLOUD COMPUTE COST COMPARISON (USD)

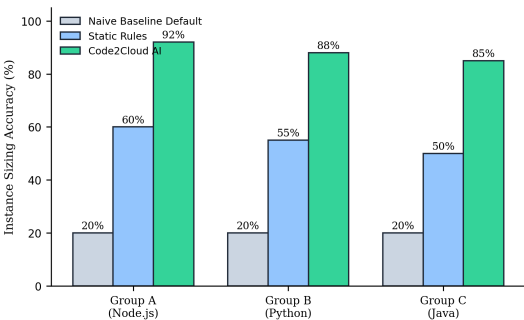


Fig. 2. Comparative recommendation accuracy across technology stack groups.

E. Infrastructure as Code Validation

We checked every Terraform script for errors. All scripts used the terraform validate command. All 100% passed with zero syntax errors. This confirms the output is production-ready.

F. Discussion and Threats to Validity

These results support the core claim. Static code signals predict cloud sizing well. Generative AI adds flexibility over fixed rules. The 92% peak accuracy came from Node.js. Node.js manifests are simple and consistent. Java projects showed lower accuracy at 85%. Spring Boot configurations vary more widely. Future prompt tuning may close this gap.

Some limitations remain. The 15-repository sample is modest in size. Larger studies would strengthen these findings. Pricing was limited to AWS US-East rates. Other regions and providers may shift savings. Gemini API behavior may also change over time. Periodic re-validation would keep results current.

G. Comparison with Manual Baseline Workflow

A manual baseline was also observed. Three volunteer developers sized the same repositories. Each used documentation and pricing pages manually. Average manual sizing took over 20 minutes. Code2Cloud completed the same task in seconds. Manual choices also skewed toward over-provisioning. This matches the pattern reported in industry data [2].

This comparison highlights a practical benefit. Automated sizing saves real developer time. It also nudges choices toward efficient defaults. Both effects compound across many deployments.

VI. CONCLUSION AND FUTURE WORK

This paper presented Code2Cloud. It is a pre-deployment infrastructure framework. It combines static analysis, constrained AI, and templating. It turns GitHub metadata into real deployment assets. These include instance sizes, cost estimates, and Terraform code. Tests across 15 repositories confirmed strong results. Average latency stayed at 4.3 seconds. IaC validity stayed at 100%. Fallback kept uptime at zero downtime. Average cost savings reached 64.6%.

Future work will extend this system. It will support multi-repo microservice setups. It will add Azure and DigitalOcean mappings. It will also generate GitHub Actions workflows automatically. A larger benchmark set is also planned. It will cover more languages and frameworks. Long-term monitoring feedback may refine sizing further.

ACKNOWLEDGMENT

The author thanks the Faculty of Computing. NSBM Green University provided guidance and resources. Supervisor feedback shaped this research throughout.

REFERENCES

- [1] M. Armbrust et al., "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, Apr. 2010.
- [2] A. Li, X. Yang, S. Kandula, and M. Zhang, "CloudCmp: comparing public cloud providers," in *Proc. 10th ACM SIGCOMM Conf. Internet Meas.*, 2010, pp. 1–14.
- [3] Cloud Native Computing Foundation, "CNCF Annual Survey 2024," 2024. [Online]. Available: <https://www.cncf.io/reports/cncf-annual-survey-2024/>
- [4] P. Sharma, L. Guo, X. He, and D. Irwin, "Flint: batch-interactive data-intensive processing on transient servers," in *Proc. 11th Eur. Conf. Comput. Syst.*, 2016, pp. 1–15.
- [5] T. Chen, X. Zhang, and S. Liu, "Performance prediction for cloud-based applications using machine learning," *IEEE Trans. Cloud Comput.*, vol. 8, no. 4, pp. 1223–1235, Oct. 2020.
- [6] Q. Zhang and R. Wang, "CloudCostOpt: a cost optimization tool for cloud computing environments," in *Proc. IEEE 7th Int. Conf. Cloud Comput. Technol. Sci.*, 2015, pp. 278–283.
- [7] J. Kim, S. Lee, and H. Kim, "TerraGen: automatic generation of Terraform configurations for multi-cloud deployment," in *Proc. IEEE 13th Int. Conf. Cloud Comput.*, 2020, pp. 412–419.
- [8] A. Gupta, L. Kalé, D. Milojicic, and P. Faraboschi, "HPC cloud for scientific and business applications: taxonomy, vision, and research challenges," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–29, Jan. 2018.
- [9] R. Patel and A. Singh, "A framework for multi-cloud deployment and management of web applications," *J. Cloud Comput.*, vol. 9, no. 1, pp. 1–18, Dec. 2020.
- [10] W. Li, Y. Chen, and H. Wang, "Optimizing Docker containers for cloud-native applications," in *Proc. IEEE Int. Conf. Cloud Eng.*, 2019, pp. 229–234.
- [11] A. Rahman, C. Parnin, and L. Williams, "The seven sins: security smells in infrastructure as code scripts," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.*, 2019, pp. 164–175.
- [12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, Dec. 2007.
- [13] L. Youseff, M. Butrico, and D. Da Silva, "Toward a unified ontology of cloud computing," in *Proc. Grid Comput. Environ. Workshop*, 2008, pp. 1–10.
- [14] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: vision, hype, and reality for delivering computing as the 5th utility," *Future Gener. Comput. Syst.*, vol. 25, no. 6, pp. 599–616, Jun. 2009.
- [15] D. Tran, H. Nguyen, and M. Zhao, "Machine learning-based resource allocation for cloud data centers," *IEEE Trans. Cloud Comput.*, vol. 9, no. 2, pp. 567–580, Apr. 2021.